

Pong Arena

Game Design and Implementation Report – Joseph Adeboye

Module	B37VB Praxis Programming	Type	Individual Game Project
---------------	--------------------------	-------------	-------------------------

1. Overview

Pong Arena is a real-time Pong game written in C. The project was designed to meet the brief of creating an original game with clear code structure, good use of variables and functions, and a walkthrough explaining the design and implementation. The game includes a live scoreboard, three AI difficulty levels, save/load support, a pause feature, and replayable one-on-one matches against the computer.

The design goal was to keep the mechanics simple enough to demonstrate good C programming fundamentals, while also adding enough features.

2. Design goals and feature set

Goal	Implementation	Benefit to the Player
Replayability	Randomized ball restart direction and three AI levels	Each match feels slightly different and the player can increase challenge
Game polish	Pause, restart, save and load commands	The player has control over the flow of the match
Clear structure	GameState, Ball and Paddle structures plus separate functions	Logic is easier to understand, test and maintain
C programming skill	Non-blocking input, file handling, loops and collision logic	Demonstrates a wide range of core programming techniques

Gameplay mock-up

[illegible]

Figure 1. Example of the board layout in the running game.

3. Program structure

The code is organised around small reusable functions. A GameState structure stores all match data in one place: ball position and speed, paddle positions, current scores, selected difficulty, pause state and round number. This keeps the program state consistent and avoids scattered global variables.

Key functions are divided by responsibility. `showMainMenu()` handles navigation, `initialiseGame()` prepares a new match, `processInput()` reads keyboard commands, `updateAi()` moves the computer paddle, `updateBall()` performs collision and scoring logic, `drawGame()` renders the board, and `saveGame()/loadGame()` manage file storage.

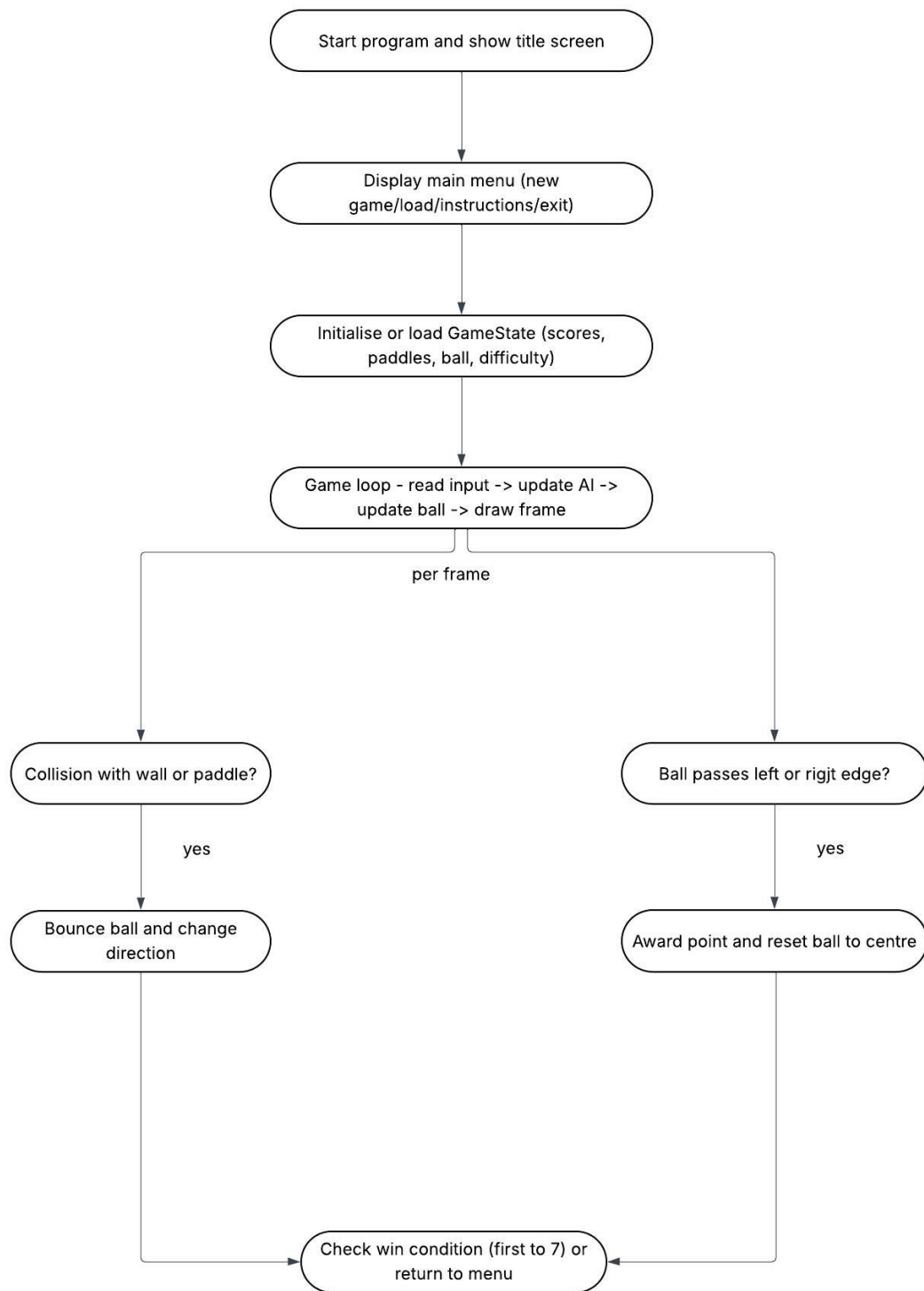


Figure 2. High-level flow of the program from menu to game loop and scoring.

4. Important implementation details

Collision and movement.

The ball stores both horizontal and vertical velocity. Each frame calculates the next position. If the next Y coordinate touches the top or bottom wall, the vertical direction is inverted. If the next X coordinate matches a paddle, the horizontal direction is reversed and the bounce angle changes depending on where the ball hit the paddle. This creates a more interesting match than simply reflecting the ball at a fixed angle.

Artificial intelligence.

The AI follows the ball but uses different reaction settings for each difficulty level. Easy mode reacts slowly and includes a larger random offset, Medium mode tracks the ball more reliably, and Hard mode reacts quickly with minimal aiming error. This provides progression without needing separate code for three different opponents.

File handling.

The save feature writes the important game values into a plain text file. When the player loads the game, those values are read back into the GameState structure. This demonstrates practical use of fopen, fprintf and fscanf in addition to real-time gameplay code.

5. How to play and testing

Item	Details
Controls	W moves the player paddle up, S moves it down. P pauses, R restarts, K saves, and Q returns to the menu.
Objective	Send the ball past the AI paddle. The first side to reach 7 points wins the match.
Compilation	Compile with a C compiler such as gcc: gcc -std=c11 pong_game.c -o pong_game
Testing carried out	Tests included paddle boundary checks, wall bounce checks, score updates, round reset, save/load verification and all three difficulty settings.

Pong Arena provides a clear design, several gameplay features and enough implementation depth to demonstrate good programming understanding.